

# YouTube, curated by parents

More radio, less browsing. Approve a collection once; children get pictures, Next and Shuffle.

### RECOMMENDATION

Build a parent-owned catalog of exact approved videos. Trial a separate YouTube category only after proving a supported Apple TV player and acceptable child-access behavior. For a strict, native tvOS experience today, rights-cleared media files delivered through Jellyfin are the strongest path.

### Yes: Jellyfin already has a web admin dashboard

Open `http://1531server.local:8096/web/` and sign in as an administrator. Library settings live under **Admin > Dashboard > Server > Libraries**. This covers the existing RAID catalog; a YouTube search, review and approval workflow would be an added admin tool, preferably a small companion web service. [S1]

| What we learned                      | What it means for this family                                                                                                                                                                                          |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Discovery is feasible                | The Data API finds playlists and returns their member video IDs. A search term is a discovery hint, never permission to keep adding matches. [S4, S5]                                                                  |
| Apple TV playback is the gate        | YouTube's supported embed is a web player. Apple's WKWebView availability excludes tvOS; our current tvOS SDK probe also failed to import WebKit. A normal embedded web player is not a ready tvOS solution. [S8, S13] |
| An embed is not a sealed kids player | rel=0 still allows same-channel recommendations. Ads and player links need separate review; whitelisting our cards does not whitelist everything inside YouTube's player. [S9, S12]                                    |
| Plugins are conditional              | YouTube metadata support exists. A third-party stream plugin uses yt-dlp and FFmpeg, which introduces a different reliability and authorization question. It is not installed or validated here. [S2, S3]              |

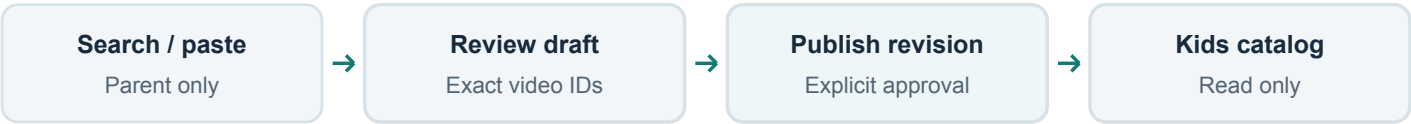
### Two choices, kept independent

**Transport:** official player, a stream resolver, an authorized mirror, or licensed files. **Presentation:** mix approved collections into Shows/Movies, or add a YouTube category. A third tab cannot solve a playback or permission problem; a working stream cannot solve weak parental approval.

Evidence: official documentation reviewed on 8 October 2026; local public-SDK import probe only. No live YouTube playback, plugin install, server change or RAID write occurred. Little Bear and Thomas are illustrative search terms, not approved sources.

# Approve the set, not the search

The parent sees discovery. The child sees only the published result.



| Admin screen | Controls and resulting behavior                                                                                                                                                                                                                                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Find      | Search Little Bear or Thomas, or paste a playlist URL. Show original YouTube titles, channels and artwork with attribution. Offer channel, language and duration filters plus preview. safeSearch=strict helps discovery; it does not establish family suitability. [S4]                                                                                |
| 2. Review    | Fetch every playlist page, then video metadata. Display channel identities, order, duration, region/embed status and unavailable entries. Parent excludes clips, compilations or unsuitable items and chooses a family label. A partial fetch cannot become a publishable draft. [S5, S6] Embed status is a hint; actual playback must still be tested. |
| 3. Publish   | Parent approves this exact list, in bulk if desired. Commit the reviewed draft hash and revision atomically. Choose category, artwork, approved Next order and Shuffle behavior. A changed draft requires renewed review.                                                                                                                               |
| 4. Maintain  | A changes inbox shows additions, removals, reorderings and metadata changes. New members stay pending. Parent can republish, hide a collection or revoke a video. No automatic channel ingestion.                                                                                                                                                       |
| 5. Kids      | One approved playlist becomes one illustrated collection card. Next and Shuffle select only admitted members. Specific-video selection is behind Parents. No search, trending feed, channel browser or arbitrary URL input.                                                                                                                             |

## What the whitelist actually identifies

Store approved video IDs, trusted channel IDs, collection UUID, ordered membership, reviewer, approval time and policy revision. The playlist URL is provenance. Matching titles, channel display names and the made-for-kids flag are not authorization. Every catalog response, image fetch, deep link and play request must pass the same membership policy.

### THE SUBTLE FAILURE

A playlist owner can add videos tomorrow. Even an approved video can be trimmed while keeping its URL. A snapshot protects membership, not immutable audiovisual content. Metadata changes trigger review, but cannot detect every content change. A stronger content guarantee requires a permitted, verified local file. [S18]

Use real Jellyfin administrator authorization for publishing. The child's local Parents PIN is a UI gate, not a server credential. Default to public playlists and server-held API credentials; no child's Google account, cookies or OAuth session.

# Official player or Jellyfin resolver?

Both can consume an approved catalog. Their playback guarantees differ sharply.

## A. Official YouTube playback

**Strawman.** Put a YouTube iframe inside the SwiftUI player, pass the approved playlist URL, set `rel=0`, and call it kid-safe. It looks like a small integration; it assumes both a tvOS web view and complete control of player navigation.

**Steelman.** On a supported web or iOS host, use the official player and load one admitted video ID at a time. Handle state, time, seek and error events; obtain fresh approval before Next or Shuffle. Keep required identification, controls and attribution. Parent previews use this route where available. [S8, S10]

**Stress test.** A full upstream playlist bypasses our frozen membership. Same-channel recommendations remain with `rel=0`, and controls cannot simply be covered to hide them. Privacy-enhanced embeds can still show ads. The public tvOS SDK lacks WebKit; a browser prototype would not prove the Apple TV app. [S9, S10, S12, S13]

**Verdict.** Good for adult preview and a conditional non-tvOS experiment. Do not promise the requested Apple TV experience until a supported player route and its escape paths are demonstrated. YouTube's living-room partner program is a device-manufacturer route, not a general native SDK for this app. [S15]

## B. Resolve YouTube streams through Jellyfin

**Strawman.** Install YouTubeSync, point it at a channel, and let periodic sync turn everything into episodes. The kids app keeps using its current VLC player. This ignores manual approval, provider changes and the server's existing access boundary.

**Steelman.** A controlled adapter consumes only our approved snapshot. A restricted worker resolves an admitted ID and exposes a short-lived playback session to the client. Generated metadata lives on the server's internal disk. The maintainer's plugin describes yt-dlp URL resolution and optional FFmpeg HLS restreaming; its default periodic sync would need replacement. [S3]

**Stress test.** Extraction can fail as YouTube changes; stream URLs expire, WAN outages remain visible and FFmpeg adds load. The plugin excludes age-restricted/member-only playback today. Downloading or bypassing the supported player is not authorized merely by holding a Data API key. [S3, S11, S17]

### VERDICT

Closest technical fit to our native Next/Shuffle experience, but not the default shipping proposal. Establish applicable permissions, server compatibility and maintenance ownership before any installation or resolver prototype. No plugin was tested against 1531Server in this research.

# Local files, familiar playback

Local playback can meet the radio-like product promise. Acquisition is a separate gate.

## C. Mirror the approved YouTube collection

**Strawman.** Run a downloader every night, mirror the whole playlist onto the RAID and let Jellyfin scan it. It seems to make YouTube reliable, but permits unreviewed additions and writes to the storage you explicitly protected.

**Steelman.** Only if the necessary permissions are established: ingest exact approved IDs into a new, bounded folder on the server's internal disk. Stage, verify and atomically publish each permitted file. Record source, approval and file digest. Publish no new playlist members automatically; retain an explicit removal and revocation policy.

**Stress test.** YouTube restricts downloading/caching audiovisual content without prior written approval; rightsholder permission also matters. Premium or an API key is not a redistribution license. Storage capacity, regional availability, upstream removal and the intended retention period need decisions. [S11, S17]

**Verdict.** Excellent playback and reboot resilience once authorized, but a conditional acquisition route. Do not build a generic YouTube download service as the first step.

## D. Acquire rights-cleared files outside YouTube

**Strawman.** Abandon the YouTube feature and ask parents to manage downloads manually. This loses the convenience of discovering a show and publishing a collection.

**Steelman.** Keep YouTube as a parent discovery and provenance source, then obtain authorized files or a licensed feed directly from a creator or distributor. Publish those files through Jellyfin after the same review. Kids retain the existing player, local artwork, progress tracking, Next and Shuffle without embedded recommendations.

**Stress test.** The exact desired shows may not offer licensed files, and a streaming subscription does not imply export rights. We have not identified an available, licensed Little Bear or Thomas catalog. Approved acquisition may be more work than watching through the official YouTube app.

**Verdict.** Strongest design for strict child containment and a native Apple TV client. It satisfies the experience when rights and files are available; it does not claim that all YouTube playlists can become Jellyfin media.

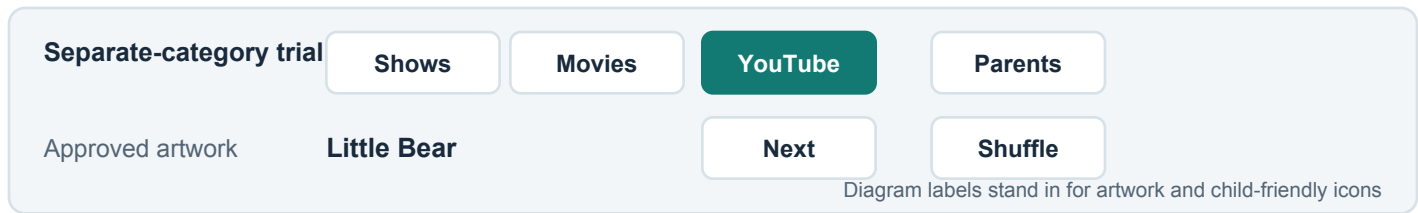
### RAID INVARIANT

No import, rename, delete, transcode output, artwork sidecar or cleanup writes on the RAID. All new state and permitted files use a dedicated internal-disk location with bounded space. New workers receive no RAID write access. Existing library additions remain an explicit parent action, never a broader drive scan.

If a new Jellyfin library is needed, migrate the client's exact library binding deliberately. The current restricted account permits exactly Kid TV and Kid Movie; enabling every folder would break that safety property.

# One familiar shelf, or a new doorway?

The source adapter and approval model stay the same under either navigation choice.



## 1. Alongside the RAID-backed Shows and Movies

**Strawman.** Merge every matching show title and hide the source. Children see one Little Bear card, while the app silently switches between Jellyfin episodes, YouTube clips and compilations.

**Steelman.** Present approved collections on the familiar shelf, with explicit parent grouping and a visible YouTube badge. Keep sources distinct underneath. Begin with one card per playlist; never deduplicate by fuzzy title alone. Next and Shuffle stay within the selected approved collection. Cross-source sequencing requires a parent-authored order.

**Tradeoff.** Fewest root choices for a non-reader and the clearest radio metaphor. It also makes WAN failures, different player controls and duplicate artwork harder to explain. A silent fallback to another source or a new search result is forbidden.

## 2. A separate YouTube category

**Strawman.** Add a red YouTube tab that opens the normal YouTube experience. This is easy to recognize but exposes the discovery interface the parent explicitly wanted outside the kids app.

**Steelman.** Add an illustrated source doorway that contains only published collection cards. A child chooses Little Bear, then Next or Shuffle. Large artwork, stable focus order and optional spoken labels support non-readers. Detailed membership and all discovery remain behind Parents.

**Tradeoff.** An extra navigation choice, but clearer source identity, outage isolation and containment testing. It can be hidden whenever no approved collection is available. The category cannot constrain YouTube's own embedded ads or recommendations.

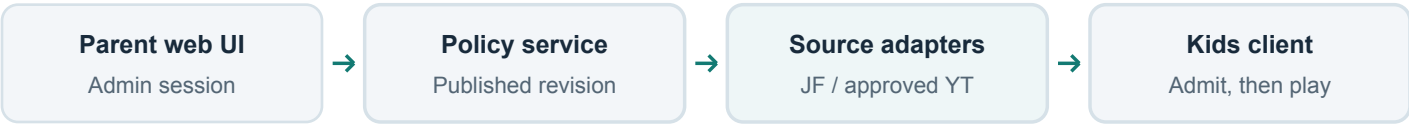
### RECOMMENDED FIRST TRIAL

Use a separate category if a live YouTube player passes the transport gates. Later, merge presentation only if measured child navigation improves and source boundaries stay clear. Authorized local files can use the existing Shows/Movies shelves immediately after a deliberate library-binding migration, if needed.

Unavailable video: say 'This show is taking a break' with Back and Retry. Skip only to another currently admitted member. An empty collection goes back to its shelf; it never opens search, an external YouTube app or a random recommendation. Advanced episode selection remains a parent feature.

# Source boundaries and shared state

Proposed components reuse the client's existing domain, catalog and playback responsibilities.



| Boundary                | Responsibility and constraint                                                                                                                                                                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server admin + policy   | Companion service on 1531Server owns drafts, approvals, audit history and revocation. Validate actual Jellyfin admin identity for writes; give the child only authenticated published-catalog and playback-admission access. Keep API keys server-side, restricted to the required API.                 |
| Client domain + catalog | Add a source-aware identity and curated-collection model in KidsDomain. Compose Jellyfin and approved-YouTube catalog adapters in KidsCatalog. A small YouTube adapter package is justified; a package per DTO is not. Preserve the exact two-library JF binding until an explicit versioned migration. |
| Playback + state        | Reuse the KidsPlaybackSession lifecycle and provide a separate player adapter with explicit seek, resume, EOF and cap capabilities. Namespace progress as jf:server:item or yt:video; keep collection and account identity separate. Do not fake TV season/episode numbers for playlist positions.      |
| Persistence             | The server is the approval authority. Existing SwiftData/iCloud state can sync the family's progress and shuffle bags after a schema migration, but cannot authorize a stale or revoked item. Track app-local viewing state; no YouTube likes, comments or watch-history writes.                        |

## Fresh approval at the point of use

Proposed API: draft preview, revision-checked publish, published catalog, admit(video ID, revision), and revoke. Admission checks membership and current availability. Use a short policy lease with a defined maximum revocation delay, then recheck before every transition. When the lease expires offline, YouTube stops; existing Jellyfin access continues independently. Stop an active revoked video at the next lease check.

## Bounded latency and failure

No YouTube search on child launch. Fetch a compact approved manifest, prefetch only admitted artwork, and isolate WAN errors from RAID playback. Validate IDs and fixed upstream hosts; accept no arbitrary resolver URLs. Live YouTube startup is bounded by WAN/player latency, so RAID seek time is not its performance target.

API budgets: current docs give search.list 100 calls/day in its separate Search Queries bucket; playlist and video metadata reads use the general quota. Verify the project's actual console limits. Refresh or delete retained public API metadata within the applicable 30-day policy window; keep parent-authored approvals separate from cached metadata. Child-directed service obligations and Apple review requirements are launch gates. [S7, S11, S14]

# Prove the gate before the feature

This document is complete research. Every integration stage below is proposed work.

| Stage / gate          | Evidence required to advance                                                                                                                                                                                         |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0. Transport decision | Supported tvOS route and applicable permissions; real Apple TV behavior for recommendations, ads, links, focus and Back. A web demo is insufficient. If strict containment fails, choose rights-cleared local media. |
| 1. Parent prototype   | Search/preview two real candidate collections, fetch all pages, review exact IDs, publish a revision and demonstrate a pending playlist addition. No actual source is approved by this document.                     |
| 2. Policy tests       | Reject partial pagination, stale draft hashes, unknown IDs, arbitrary URLs and new members. Exercise revocation, unavailable videos, unapproved artwork, account switching and restored old caches.                  |
| 3. End-to-end trial   | On Apple TV: Next, Shuffle, EOF, progress resume, sleep, WAN loss and reboot. Verify session caps, error exits and no unintended source transition. Simulated iCloud sync is not live-device proof.                  |
| 4. Operate + release  | Internal-disk backups, bounded storage, restart/backoff and quota monitoring; demonstrate no RAID writes. Complete child-directed privacy and platform review before distribution. [S11, S14, S16]                   |

**Success:** no unapproved item can be selected through our UI, API or restored state; a parent can see and revoke the exact set; a missing YouTube source does not break Jellyfin. Separately measure all content and navigation exposed by the actual player. If that exceeds the family's allowed boundary, the live route fails acceptance.

## Sources and verification

- [S1] Jellyfin: library administration

[S2] Jellyfin: plugins and YouTube metadata

[S3] YouTubeSync: maintainer's implementation

[S4] YouTube Data API: search.list

[S5] YouTube Data API: playlistItems.list

[S6] YouTube Data API: video metadata

[S7] YouTube Data API: quota and setup

[S8] YouTube: IFrame Player API

[S9] YouTube: player parameters
- [S10] YouTube: minimum player requirements

[S11] YouTube: developer policies

[S12] YouTube: embedding and privacy

[S13] Apple: WKWebView platform availability

[S14] Apple: App Review Guidelines

[S15] YouTube: living-room device partners

[S16] YouTube: policy implementation guide

[S17] YouTube: service terms

[S18] YouTube: editing without changing the URL

Local evidence: docs/design/youtube-tvos-feasibility.json, observed 8 October 2026. AppleTVSimulator27.0 SDK, arm64 tvOS simulator target: public WebKit framework absent; 'import WebKit' failed. This is SDK feasibility evidence only, not a full player test. Current client code was inspected for exact library binding, item identity and playback lifecycle. All uncited system behavior in this document is a design proposal, not an implemented feature.